Copyright All rights reserved

Network Programming - 04

afewhee@gmail.com

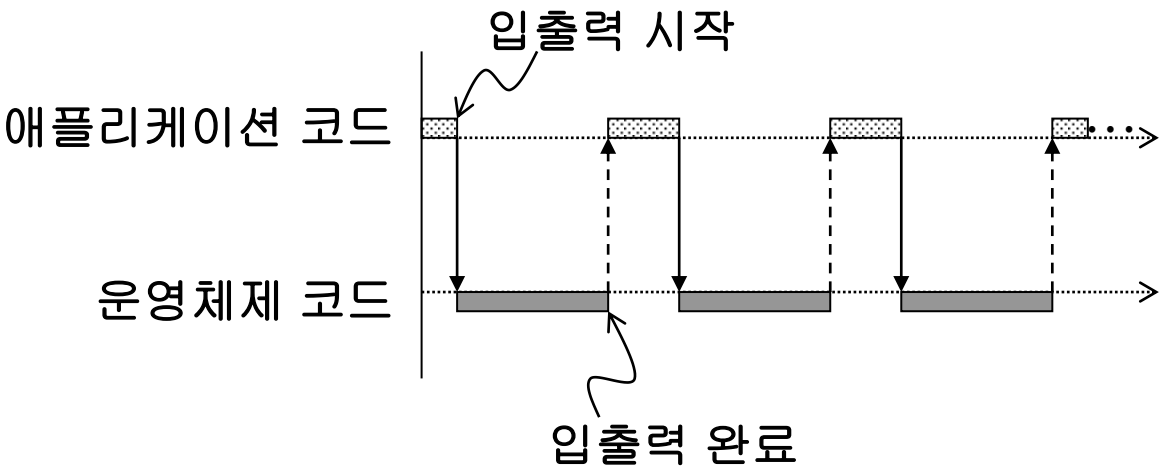
0. 목차

Heesung Oh
Copyright All rights reserved

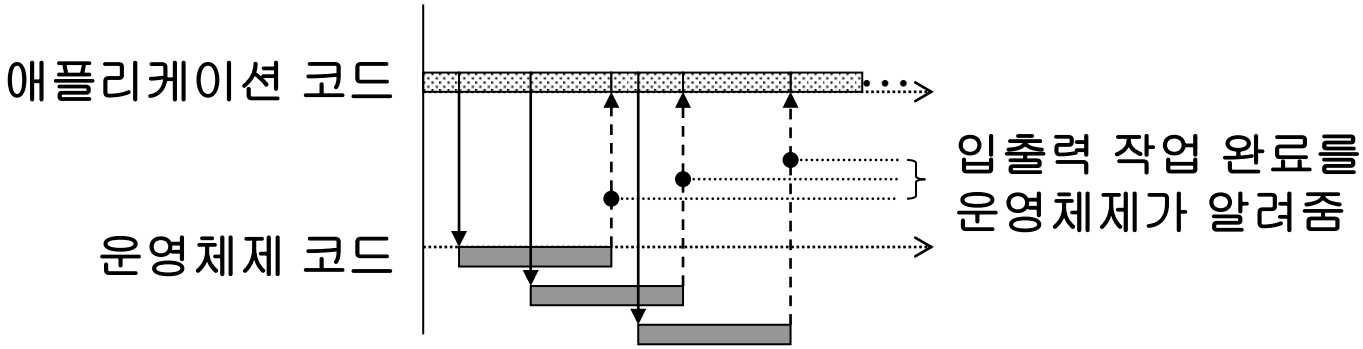
- I/O 모델
- IOCP
- 네트워크 Util
  - 메시지 큐, 링 버퍼
  - 패킷 디자인

Heesung Oh
Copyright All rights reserved

- **동기식(Synchronous) 입출력**



- **비동기식(Asynchronous) 입출력**



1. I/O 모델: 동기식-비동기식

Heesung Oh
Copyright All rights reserved

- 동기식 입출력(synchronous I/O)
  - 애플리케이션은 입출력 함수를 호출한 후 입출력 작업이 끝날 때까지 대기
  - 입출력 작업이 끝나면 입출력 함수는 리턴하고 애플리케이션은 입출력 결과를 처리하거나 다른 작업을 진행

- 비동기식 입출력(asynchronous I/O)
  - 윈도우의 Overlapped I/O
  - 애플리케이션은 입출력 함수를 호출한 후 입출력 작업의 완료 여부와 무관하게 다른 작업을 진행
  - 입출력 작업이 끝나면 운영체제는 작업 완료를 애플리케이션에게 알려줌. 이때 애플리케이션은 다른 작업을 중단하고 입출력 결과를 처리

  - 윈도우의 특화 ➔ IOCP

Heesung Oh
Copyright All rights reserved

입출력 방식에 따른 소켓 입출력 모델 분류

- 동기 입출력 + 비동기 통지
  - Select 모델, WSAAsyncSelect 모델, WSAEventSelect 모델
- 비동기 입출력 + 비동기 통지
  - Overlapped 모델(I), Overlapped 모델(II), Completion Port 모델

1. I/O 모델: 블로킹/넌-블로킹, 동기식/비동기식

Heesung Oh
Copyright All rights reserved

- Blocking / Non-blocking
  - 소켓 함수 호출 후 제어권 반환 방식

- 동기 I/O 모델
  - 입출력 요청이 순차적으로 처리되고
  - 결과에 의존해서 다음 작업이 진행되면

- 비동기 I/O 모델
  - 입출력 요청을 중첩해서 동시에 걸어두고
  - 완료를 나중에 각각 처리하면

Heesung Oh
Copyright All rights reserved

- Overlapped 모델 사용 절차
  - 비동기 입출력을 지원하는 소켓 생성
  - 비동기 입출력을 지원하는 소켓 함수 호출
  - 운영체제가 소켓 입출력 작업 완료를 애플리케이션에게 비동기로 통지
  - 애플리케이션은 결과를 처리

1. I/O 모델 – Overlapped I/O 모델 분류

Heesung Oh
Copyright All rights reserved

- Overlapped 모델(I) – 이벤트 방식
  - I/O 완료와 바인딩 된 이벤트 객체를 이용해서 애플리케이션이 이벤트 객체로 작업 완료를 감지.

- Overlapped 모델(II) – 완료 루틴 방식
  - 소켓 입출력 작업이 완료되면, 콜백 함수(callback function)로 구현된 완료 루틴 함수를 자동으로 호출

- Overlapped 모델(III) – IOCP
  - I/O 완료를 담당하는 Completion Port를 이용한 모델
  - IOCP: 비동기 입출력 결과와 이 결과를 처리할 스레드에대한 정보를 담고 있는 구조
  - 특징: Overlapped I/O + 비동기 Notification I/O
  - 장점: 제한된 쓰레드(Work Thread)를 통해서 여러 소켓의 입출력 처리. 컨텍스트 스위칭에 소비되는 시간을 줄임

2.1 Overlapped I/O

Heesung Oh
Copyright All rights reserved

- Overlapped 모델
  - 이전 입출력 미완료 상태에서도 다음 입출력 요청 가능
  - 동시 입출력 진행 방식

- BSD 소켓 함수 I/O
  - send(): 송신 버퍼 → 소켓 전송 버퍼 복사
  - recv():소켓 수신 버퍼 → 사용자 버퍼 복사

- 중첩 I/O 전용 함수
  - WSASend()
  - WSARecv()

2.2 Overlapped I/O – 주의 사항

Heesung Oh
Copyright All rights reserved

- I/O 버퍼
  - 입출력 완료 전 버퍼 변경 금지
  - 지역 변수 버퍼 사용 → 스택 해제 → 오류 발생
- 동일 I/O버퍼
  - 동일 버퍼 연속 송신 → 송신 미완료 상태 데이터 변경
  - 수신 데이터 불일치 가능
- OVERLAPPED
  - 입출력 완료 시까지 유지 필수
  - 송신 / 수신 구조체 분리 ← 재사용 금지

2.3 Overlapped I/O - 이벤트 방식 File Read 준비물

Heesung Oh
Copyright All rights reserved

- OVERLAPPED 구조체
  - `OVERLAPPED g_rOL;`
- Event 객체: 자동 리셋 / non-signaled
  - `CreateEvent(nullptr, FALSE, FALSE, nullptr);`
- Read Buffer
  - `char g_rBuf[MAX_BUF+4];`
- File Handle:
  - `CreateFile(… , FILE_ATTRIBUTE_NORMAL|FILE_FLAG_OVERLAPPED);`

Heesung Oh
Copyright All rights reserved

ReadFile

```
// 비동기 read 요청
hr = ReadFile(g_hFile, g_rBuf, MAX_BUF, &dTran, &g_rOL);
```

```
DWORD WINAPI WorkThread(void* pParam) {
  int     hr   = 0;
  DWORD   dTran= 0;

  while(1)
  {
    // 입출력 완료를 기다림
    hr = WaitForSingleObject(g_rOL.hEvent, INFINITE);
    // ...
    // 완료 결과 해석. 전송된 길이 얻기
    hr = GetOverlappedResult(g_hFile, &g_rOL, &dTran, FALSE);
    // ...
    // 비동기 read 요청
    hr= ReadFile(g_hFile, g_rBuf, MAX_BUF, &dTran, &g_rOL);
    // ...
```

3.2 Overlapped I/O - 이벤트 방식 I/O

Heesung Oh
Copyright All rights reserved

- WSAOVERLAPPED
  - OVERLAPPED 와 동일
  - 비동기 I/O 상태 관리 구조체
- WSABUF
  - 윈속 비동기 I/O 전용 구조체
  - 데이터 버퍼 시작 주소
  - 데이터 길이 정보
- WSACeateEvent (수동 리셋 이벤트)
  - 수동 리셋 이벤트 생성
- ※ 참고
  - 자동 리셋
    - Wait 반환 시 자동 비신호 상태 전환
    - 미처리 I/O 존재 가능
  - 수동 리셋
    - ResetEvent 호출 전까지 신호 상태 유지
    - 여러 개 완료 I/O 확인 가능
    - 목적: Overlapped I/O 완료 누락 방지

Heesung Oh
Copyright All rights reserved

- 이벤트 객체 기반 완료 통지 구조
  - 입출력 요청 시 이벤트 객체 연결
  - 완료 시 이벤트 신호 상태 변경

- 소켓 생성

```
scHost = WSASocket(
    AF_INET,
    SOCK_STREAM,
    IPPROTO_TCP,
    nullptr,
    0,
    WSA_FLAG_OVERLAPPED
);
```

Heesung Oh
Copyright All rights reserved

- WSABUF
  - WinSocket2 데이터 버퍼 관리용 구조체
  - 버퍼 주소, 길이 포함
  - 실제 데이터가 저장된 버퍼의 위치와 크기를 OS에 전달
  - 비연속 메모리 처리 (Gather / Scatter) 용

```
typedef struct  WSABUF {
    ULONG len;     //버퍼의 길이
    CHAR* buf;     // 데이터 메모리 주소
};
```

3.5 Overlapped I/O - OVERLAP_EX

Heesung Oh
Copyright All rights reserved

- WSAOVERLAPPED
  - OVERLAPPED 와 동일
- OVERLAP_EX
  - WSAOVERLAPPED 확장
  - WSABUF, Transferred, send/recv flag, csBuf 포함

```
struct OVERLAP_EX : public WSAOVERLAPPED
{
    DWORD       dTran;                  // Transfered
    DWORD       dFlag;                  // Flag
    WSABUF      wsBuf;
    char        csBuf[MAX_BUF+4];

    OVERLAP_EX()
    {
        memset(this, 0, sizeof(OVERLAP_EX) );
        wsBuf.len = MAX_BUF;
        wsBuf.buf = csBuf;
    }
```

3.6 Overlapped I/O - 이벤트 방식 I/O 처리

Heesung Oh
Copyright All rights reserved

- 이벤트: WSAWaitForMultipleEvents
- 비동기 I/O 결과: WSAGetOverlappedResult

```
nE = WSAWaitForMultipleEvents(...);

nE -= WSA_WAIT_EVENT_0;                // 인덱스 재조정

for(i= nE; i<OVL_TOT; ++i)
{
    hr = WSAWaitForMultipleEvents(1, &vEvn[i], TRUE, 0, FALSE);
    // 비동기 입출력 결과 확인
    hr = WSAGetOverlappedResult(...);
    g_ol[i].ResetEvent();              // 이벤트 비 신호 상태


    if(OVL_CONNECT == i)               // Connect Notify
    {
    }
    else if(OVL_SEND == i)             // Send Notify
    {
    }
    else if(OVL_RECV == i)             // Recv Notify
    {
        hr = ReadFile(…);
    }
```

Heesung Oh
Copyright All rights reserved

- Send

```
hr = WriteFile((HANDLE)g_scHost // 송신 소켓
      , &g_ol[OVL_SEND].csBuf    // 송신 버퍼 포인터
      , iLen
      , &g_ol[OVL_SEND].dTran
      , &g_ol[OVL_SEND]          // OVERLAPPED 구조체 포인터
      );
```

- Recv

```
hr = ReadFile((HANDLE)g_scHost  // 수신 소켓
      , g_ol[OVL_RECV].csBuf     // 수신 버퍼
      , MAX_BUF
      , &g_ol[OVL_RECV].dTran
      , &g_ol[OVL_RECV]          // OVERLAPPED 구조체 포인터
      );
```

Heesung Oh
Copyright All rights reserved

- WSASend

```
hr = WSASend(g_scHost           // 송신 소켓
     , &g_ol[OVL_SEND].wsBuf    // WSBUF 포인터
     , 1                        // WSBUF의 수
     , &g_ol[OVL_SEND].dTran
     ,  g_ol[OVL_SEND].dFlag
     , &g_ol[OVL_SEND]          // OVERLAPPED 구조체 포인터
     , nullptr  );
```

- WSARecv

```
hr = WSARecv(g_scHost           // 수신 소켓
     , &g_ol[OVL_RECV].wsBuf    // WSBUF 포인터
     , 1                        // WSBUF의 수
     , &g_ol[OVL_RECV].dTran
     , &g_ol[OVL_RECV].dFlag
     , &g_ol[OVL_RECV]          // OVERLAPPED 구조체 포인터
     , nullptr  );
```

4.1 Overlapped I/O - 완료 루틴 File Read

Heesung Oh
Copyright All rights reserved

- OVERLAPPED, Buffer
  - 이벤트 방식과 동일하게 유지

- 완료 루틴 콜백 함수 추가
  - I/O 완료 콜백 함수

```
void  CALLBACK  CompletionRoutine(DWORD, DWORD, LPOVERLAPPED);
```

- ReadFile

```
hr= ReadFileEx(g_hFile, g_rBuf, MAX_BUF
                        , &g_rOL, CompletionRoutine);
```

4.2 Overlapped I/O - 완료 루틴 File Read Callback

Heesung Oh
Copyright All rights reserved

```
// Completion Routine
void CALLBACK CompletionRoutine(DWORD dErr, DWORD dTran, LPOVERLAPPED pOl)
{
    int hr=0;
     printf(g_rBuf);
    g_TotalRead += dTran;

    if(g_TotalSize <= g_TotalRead)
    {
        if(g_hFile)
            CloseHandle(g_hFile);
         g_hFile = 0;
        return ;
    }

     // Read file 재 요청
    g_rOL.Offset += dTran;
     hr = AsyncRead();
     Sleep(30);
     return;
}
```

4.3 Overlapped I/O - 완료 루틴 I/O

Heesung Oh
Copyright All rights reserved

- typedef LPWSAOVERLAPPED_COMPLETION_ROUTINE PWSA_COMP_R;

- OVERLAP_EX

```
struct OVERLAP_EX : public WSAOVERLAPPED
{
    DWORD       dType;
    DWORD       dFlag;
    DWORD       dTran;
    WSABUF      wsBuf;
    char        csBuf[MAX_BUF+4];

    PWSA_COMP_R pcsFunc;
```

- Recv

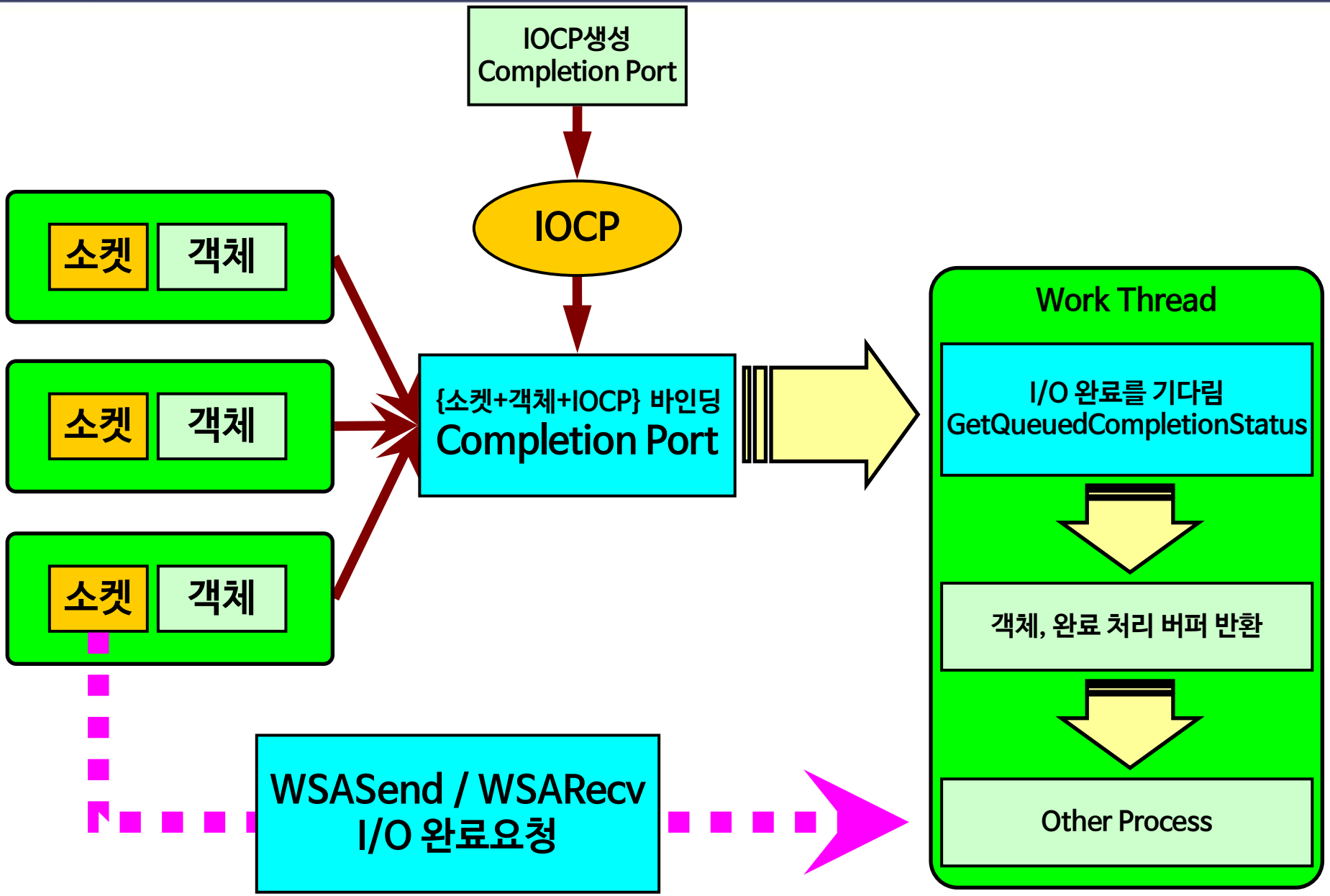
```
WSARecv(s, &wsBuf, 1, &dTran, &dFlag, this, pcsFunc);
```

- Send

```
WSASend(s, &wsBuf, 1, &dTran, dFlag, this, pcsFunc);
```

5. IOCP – 구현원리

Heesung Oh
Copyright All rights reserved



Heesung Oh
Copyright All rights reserved

- IOCP 객체 생성

```
// IOCP 객체 생성
g_hIocp = CreateIoCompletionPort(INVALID_HANDLE_VALUE, nullptr, 0, 0);
```

- I/O 용 Work Thread 생성
  - 논리적 cpu *1.5;

```
int nWrkPrc = 2 * GetSystemProcessorCount();
for(int i=0; i<nWrkPrc; ++i)
{
    HANDLE hThWrk = (HANDLE)_beginthreadex(nullptr, 0
                    , (unsigned (__stdcall*)(void*))WorkThread, nullptr, 0, nullptr);
```

- IOCP 바인딩

```
ULONG_PTR pIoKey = (ULONG_PTR)&g_hFile;                // IO Key is File
HANDLE    hRet   =
CreateIoCompletionPort(g_hFile, g_hIocp, pIoKey, 0);  // <IOCP, File, key> 바인딩
```

Heesung Oh
Copyright All rights reserved

- Read File (비동기 read 요청)

```
hr = ReadFile(g_hFile, g_rBuf, MAX_BUF, &dTran, &g_rOL);
```

- WorkThread

```
hr = GetQueuedCompletionStatus(
        g_hIocp                     // Completion Port
      , &dTran                      // 전송 된 바이트 수
      , (PULONG_PTR)&pIoKey         // 완료키 주소
      , (LPOVERLAPPED*)&pOL         // OVERLAPPED 구조체
      , INFINITE
      );

// ...
// 비동기 read 요청
hr = ReadFile(g_hFile, g_rBuf, MAX_BUF, &dTran, &g_rOL);
```

5. IOCP – I/O구현

Heesung Oh
Copyright All rights reserved

```
RemoteHost* pRmH = new RemoteHost;

CreateIoCompletionPort((HANDLE)scH
, hIocp
, (DWORD)pRmH
, 0);
```

```
pRmH->AsyncRecv();
```

```
RemoteHost* pRmH        = nullptr;
OVERLAPPED_EX* pOL      = nullptr;
DWORD dTran     = 0;
...
hr = GetQueuedCompletionStatus(
        hIocp
        , &dTran
        , (LPDWORD)&pRmH
        , (LPOVERLAPPED*)&pOL
        , INFINITE);
```

```
struct OVERLAPPED_EX
{
    OVERLAPPED OL;
    DWORD  dTran; // Transfered
    DWORD dFlag; // Flag
    WSABUF WsBuf;
    ...};
```

```
struct RemoteHost
{
    SOCKET  scH;      // 소켓
    OVERLAPPED_EX   olSnd;  // For WSASend
    OVERLAPPED_EX   olRcv;  // For WSARecv
    ...
    INT AsyncSend(char* sBuf/*in*/, int*
iSize){ ...
        hr = WSASend( scH // 송신 소켓
        , &(olSnd.WsBuf)    // 송신 버퍼 포인터
        , 1 // 송신 버퍼의 수
        , &olSnd.dTran, olSnd.dFlag
        , (OVERLAPPED*)&(olSnd) , nullptr);
        ...}

    INT AsyncRecv() {...
        hr = WSARecv( scH // 수신 소켓
        , &(olRcv.WsBuf)    // 수신 버퍼 포인터
        , 1 // 수신 버퍼의 수
        , &olRcv.dTran, &olRcv.dFlag
        , (OVERLAPPED*)&(olRcv) , nullptr);
        ...}
};
```

5. IOCP – 소켓 입출력 절차1

Heesung Oh
Copyright All rights reserved

- I/O 완료 포트 생성
  - CreateIoCompletionPort()
- Work Thread 생성
  - CPU에 맞게 적절히 생성 → 보통 1~2배
  - GetQueuedCompletionStatus() 함수를 호출, 대기 상태로 만듬
- 중첩된 비동기 입출력 소켓 생성
  - WSASocket(..., WSA_FLAG_OVERLAPPED)

5. IOCP – 소켓 입출력 절차2

Heesung Oh
Copyright All rights reserved

- {소켓 + IOCP + 객체} 바인딩
  - 비동기 입출력 결과는 입출력 완료 포트에 저장됨
  - CreateIoCompletionPort()
- {소켓 + 완료 처리 버퍼} 비동기 요청
  - WSASend(), WSARecv()
  - 비동기 입출력 작업이 곧바로 완료되지 않으면, 소켓 함수는 오류를 리턴 →
  - 오류 코드는 WSA_IO_PENDING으로 설정
- {객체 + 완료 처리 버퍼} 결과를 비동기 완료 반환:
  - GetQueuedCompletionStatus() 함수
  - 객체, 완료 처리 버퍼 등을 반환

Heesung Oh
Copyright All rights reserved

● 논리 프로세서 수

```
int GetSystemProcessorCount()
{
    SYSTEM_INFO SystemInfo;
    GetSystemInfo(&SystemInfo);
    return (int)SystemInfo.dwNumberOfProcessors;
}
```

Heesung Oh
Copyright All rights reserved

- Work Thread 생성
  - 경험적 ==〉 CPU * 1~2

```
// 2 * processor 개수만큼 Work 쓰레드 생성
int nWrkPrc = 2 * GetSystemProcessorCount();

for(int i=0; i<nWrkPrc; ++i)
{
  auto hThWrk = (HANDLE)_beginthreadex(nullptr, 0
                , WorkThread
                , nullptr, 0, nullptr);
  CloseHandle(hThWrk);
}
```

5. IOCP – 주요함수

Heesung Oh
Copyright All rights reserved

- 중첩된 비동기 입출력 소켓 생성
  - WSASocket(PF_INET, SOCK_STREAM, 0, nullptr, 0, WSA_FLAG_OVERLAPPED);

- IOCP 객체 생성
  - hIocp = CreateIoCompletionPort(INVALID_HANDLE_VALUE, nullptr, 0, 0);

5. IOCP – 주요함수

Heesung Oh
Copyright All rights reserved

- 입출력 완료 포트 생성: {소켓 + IOCP + 객체} 바인딩
  - HANDLE CreateIoCompletionPort (
    HANDLE FileHandle, // IOCP와 바인딩할 소켓
    HANDLE ExistingCompletionPort, // IOCP 핸들
    ULONG_PTR CompletionKey, // I/O를 요청한 객체 또는 완료를 구분해주는 객체
    DWORD NumberOfConcurrentThreads // 동시에 실행될 수 있는 작업 스레드 수 = 보통 0
    );
  - 성공: 입출력 완료 포트 핸들, 실패: nullptr

- 전송에 대한 비동기 요청: {소켓 + 송신 처리 버퍼} 비동기 요청
  - int WSASend (SOCKET s, // 송신 버퍼 소켓
    LPWSABUF lpBuffers, // 송신 버퍼 포인터
    DWORD dwBufferCount, // 송신 버퍼 수. 보통 1
    LPDWORD lpNumberOfBytesSent, // 송신 버퍼 크기 ➔ 계산해서 전달
    DWORD dwFlags, // 0으로 설정
    LPWSAOVERLAPPED lpOverlapped, // OVERLAPPED 구조체 포인터
    LPWSAOVERLAPPED_COMPLETION_ROUTINE lpCompletionRoutine // IOCP에서는 nullptr
) ;
  - 성공: 0, 실패: SOCKET_ERROR

- 수신에 대한 비동기 요청: {소켓 + 수신 처리 버퍼} 비동기 요청
  - int WSARecv (SOCKET s, // 수신 소켓
    LPWSABUF lpBuffers, // 수신 버퍼 포인터
    DWORD dwBufferCount, // 수신 버퍼 수, 보통 1
    LPDWORD lpNumberOfBytesRecvd, // 0으로 설정 DWORD dTran= 0; &dTran
    LPDWORD lpFlags, // 0으로 설정 DWORD dFlag= 0; &dFlag
    LPWSAOVERLAPPED lpOverlapped, // 보통 OVERLAPPED 구조체 포인터로 캐스팅
    LPWSAOVERLAPPED_COMPLETION_ROUTINE lpCompletionRoutine // IOCP에서는 nullptr
) ;
  - 성공: 0, 실패: SOCKET_ERROR

5. IOCP – 주요함수

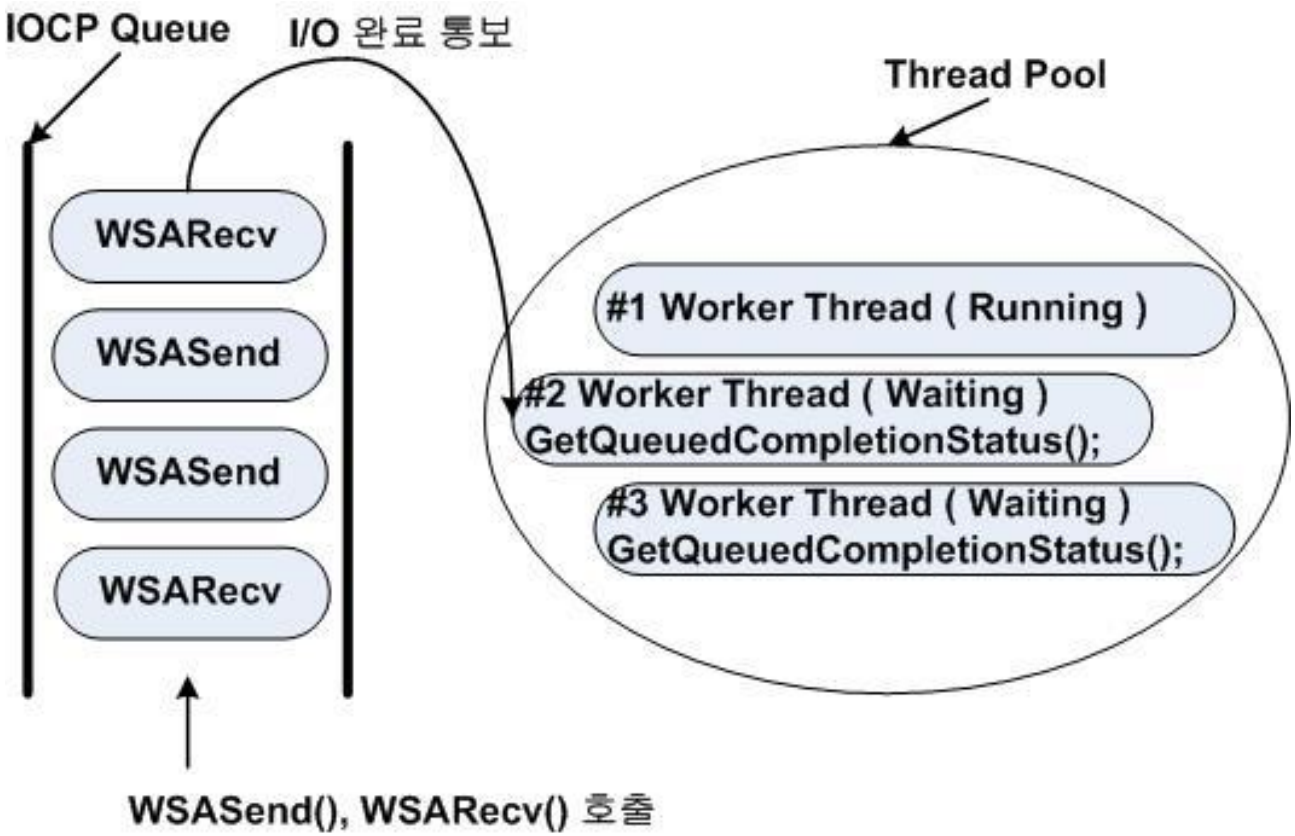
Heesung Oh
Copyright All rights reserved

- 비동기 입출력 결과 확인: {객체 + 완료 처리 요청 버퍼} 결과 비동기로 반환
  - BOOL GetQueuedCompletionStatus (
    HANDLE CompletionPort, // IOCP 핸들
    LPDWORD lpNumberOfBytes, // 완료된 I/O의 총 바이트 수
    PULONG_PTR lpCompletionKey, // I/O를 요청한 객체
    LPOVERLAPPED *lpOverlapped, // Overlapped 구조체의 주소
    DWORD dwMilliseconds // 타임아웃 INFINITE ➔ I/O 완료까지 기다림
    );
  - 성공: 0이 아닌 값, 실패: 0
  - 쓰레드에서 호출
  - 완료 결과를 IOCP 내의 I/O 완료 패킷 큐로부터 하나씩 꺼내옴

- IOCP Queue에 직접 데이터를 넣는 경우:
  - BOOL PostQueuedCompletionStatus (
    HANDLE CompletionPort, // IOCP 핸들
    DWORD dwNumberOfBytesTransferred, // bytes transferred
    ULONG_PTR dwCompletionKey, // completion key
    LPOVERLAPPED lpOverlapped // overlapped buffer
    );
  - 성공: 0이 아닌 값, 실패: 0
  - 작업 스레드에게 I/O 완료 처리와 상관없이 메시지 전달할 때 사용

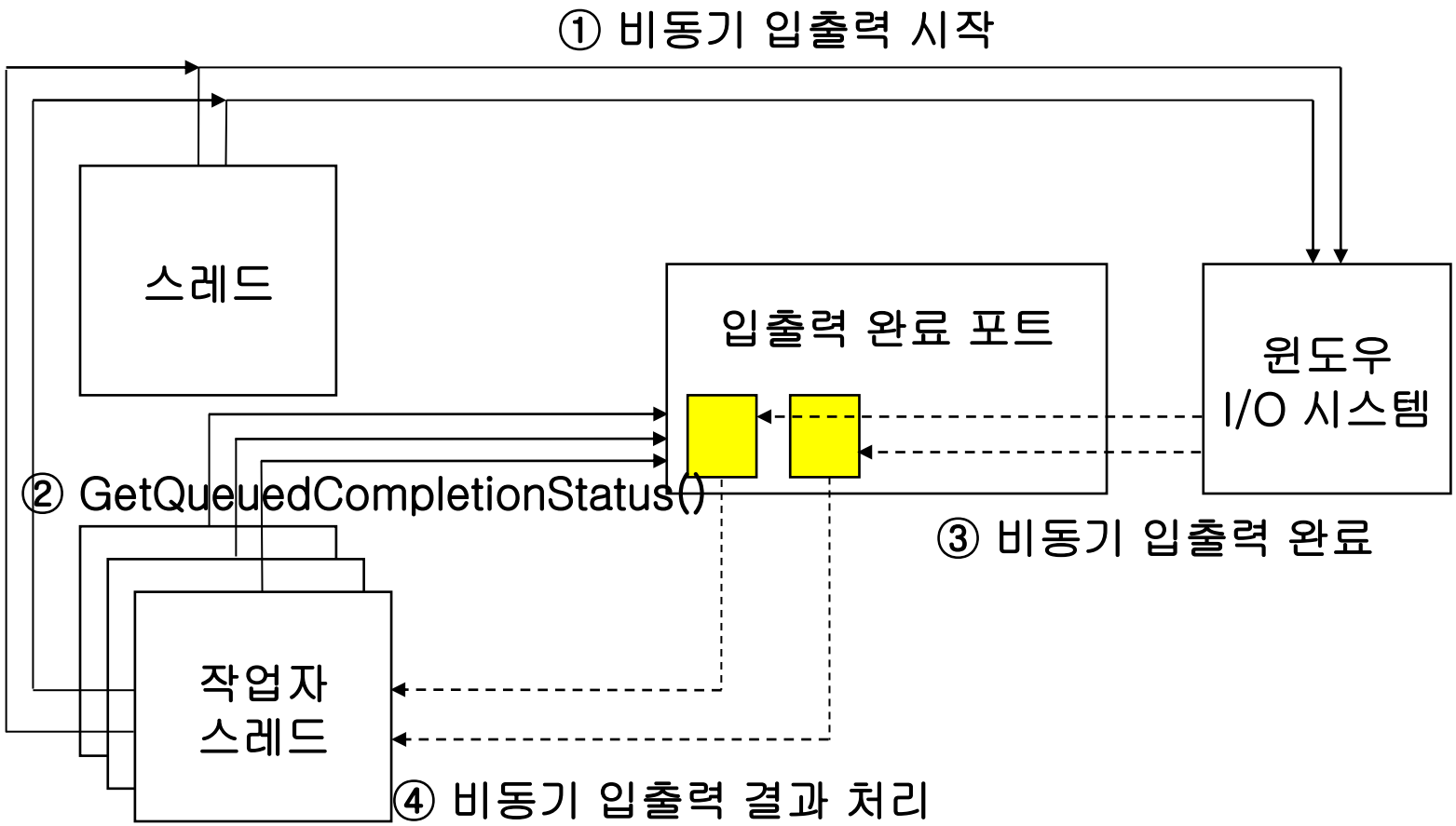
Heesung Oh
Copyright All rights reserved

Completion Port 모델 동작 원리



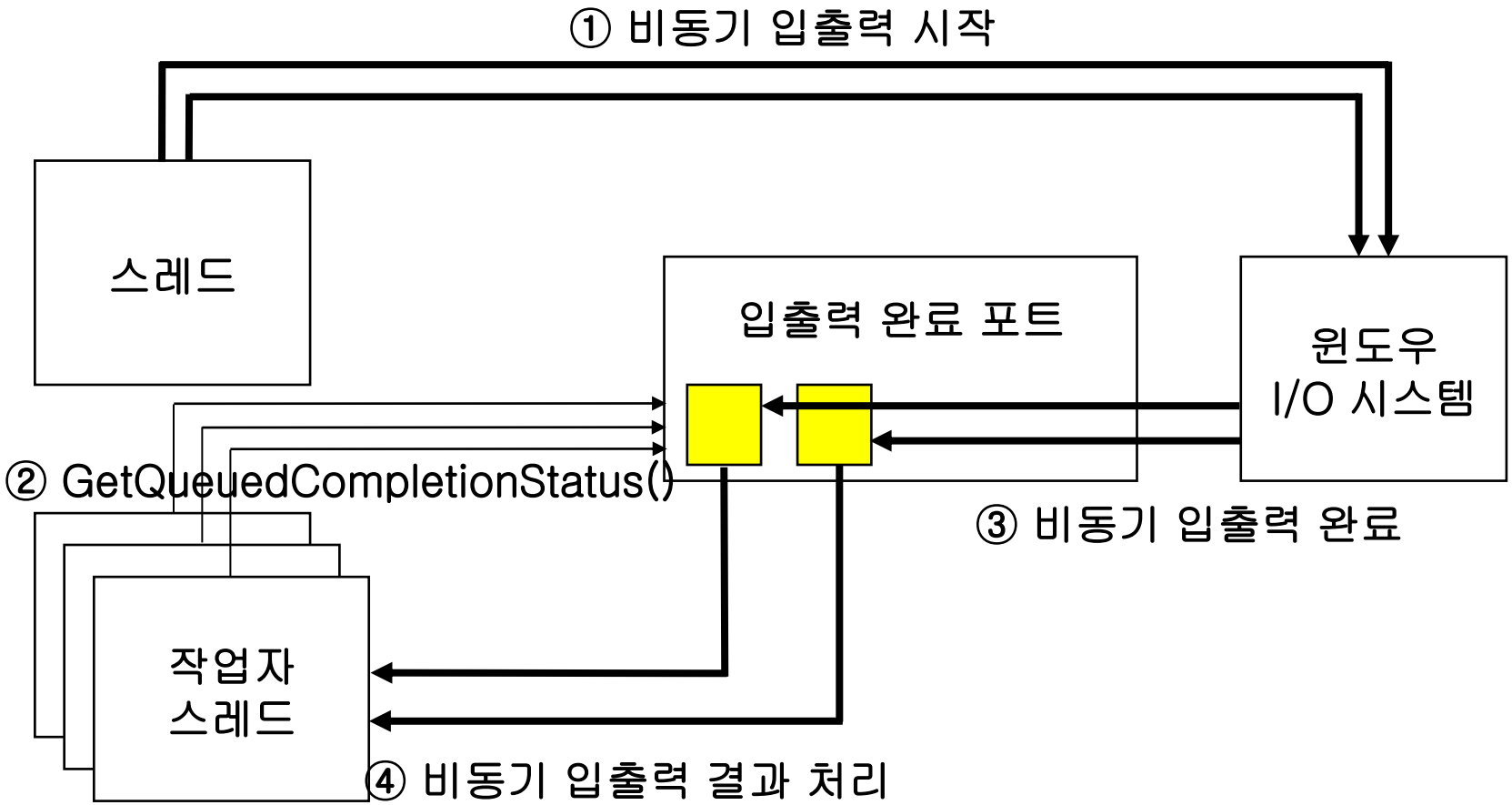
Heesung Oh
Copyright All rights reserved

Completion Port 모델 동작 원리



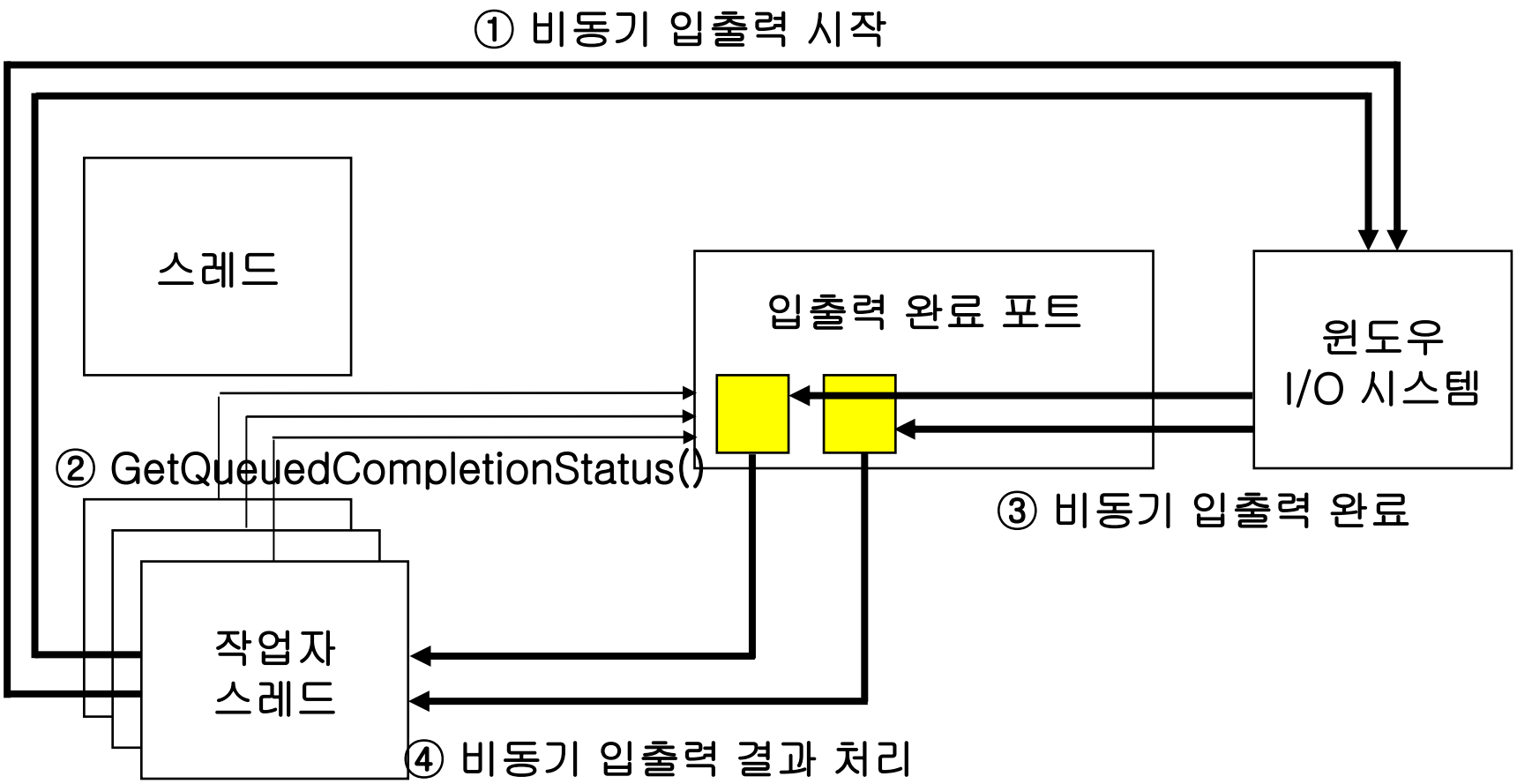
Heesung Oh
Copyright All rights reserved

Completion Port 모델 동작 원리



Heesung Oh
Copyright All rights reserved

Completion Port 모델 동작 원리



3. 네트워크 Util – 원형 큐

Heesung Oh
Copyright All rights reserved

```
template<class T>
class TqueCircular
{
protected:
    T*      m_pBuf;         // 큐의 버퍼
    int     m_iFront;       // 시작 위치
    int     m_iRear;        // 끝 위치
    int     m_iSpace;       // 남아있는 큐의 버퍼 크기
    int     m_iWidth;       // 큐의 버퍼 크기
public:
    TqueCircular();
    TqueCircular(int iWidth);
    virtual ~TqueCircular();
    int     Enqueue(T* p);  // 데이터 추가
    int     Dequeue(T* p);  // 데이터 꺼냄
};


template<class T>
TqueCircular<T>::TqueCircular()
...
```

3. 네트워크 Util – 링 버퍼

Heesung Oh
Copyright All rights reserved

```
typedef     unsigned char          BYTE;

class CQueStr
{
protected:
    BYTE* m_sBuf;                          // 큐의 버퍼
    int   m_iFront;                        // Front 위치
    int   m_iRear;                         // Rear 위치위치
    int   m_iSpace;                        // 남아 있는 공간
    int   m_iWidth;                        // 큐의 버퍼 크기

public:
    CQueStr();
    CQueStr(int iWidth);
    virtual ~CQueStr();

    int   Enqueue(void* sIn,int iSize);    // 데이터 추가기. 리턴 값은 추가 못한 나머지 사이즈
    int   Dequeue(void*sOut,int* iSize);   // 데이터 가져오기. 리턴 값은 꺼내온 사이즈

    void  Clear();                         // 큐의 내용을 지움
    int   GetWidth();                      // 큐의 크기를 가져옴
    int   GetRemain();                     // 큐의 남아 있는 크기를 가져옴
    void  Resize(int iSize);               // 큐의 사이즈를 다시 설정
};
```

Heesung Oh
Copyright All rights reserved

- 단순 패킷 구조

길이(2Byte)	메시지

- 패킷 종류가 들어간 구조

길이(2Byte)	종류(4Byte)	메시지

- End Marker가 들어간 구조

길이(2Byte)	종류(4Byte)	메시지	End Marker(4Byte)